Compilador de un DSL para simular combates RPG por turnos

Informe final — Stage III (Backend)

Autómatas, Teoría de Lenguajes y Compiladores
Ingeniería Informática — ITBA

Grupo	G-138
Integrantes	Alvaro Capridi (legajo 64.206) Tiziano Ametller (legajo 65.004)
Repositorio	https://github.com/TizianoAmetller/Flex-Bison-Compiler
Rama	`development`
Fecha	15 de noviembre de 2026

El código fuente, los programas de ejemplo y la suite de pruebas están en el repositorio; el README describe cómo construir y ejecutar el compilador.

Tabla de Contenidos

1 Introducción **1**

2 Modelo Computacional **1**
2.1 Dominio 1
2.2 Lenguaje 1

3 Implementación **2**
3.1 Frontend 2
3.2 Backend 3
3.3 Adicionales 4
3.4 Dificultades Encontradas 5

4 Futuras Extensiones **6**

5 Conclusiones **6**

6 Referencias **7**

7 Bibliografía **7**

1 Introducción

Este informe documenta el compilador que construimos para el Proyecto Especial de la materia [1]. El lenguaje, un DSL para describir combates de rol por turnos, se diseñó en el Stage I [2]; el frontend (análisis léxico y sintáctico con Flex y Bison) es el Stage II; el Stage III agrega el análisis semántico, la simulación del combate y la generación del resultado.

Un programa declara unidades, habilidades, efectos, equipos, un campo de batalla y un combate. El compilador lo valida, **ejecuta la batalla** que describe y la escribe como un registro de texto o como una página HTML autocontenida que la reproduce con métricas. Todo está escrito en C con Flex y Bison, sin bibliotecas externas.

El código, los programas de ejemplo y las pruebas están versionados en el repositorio (rama `development`), y el README explica cómo construirlo, ejecutarlo y probarlo. Aquí sólo documentamos las decisiones relevantes.

2 Modelo Computacional

2.1 Dominio

Se modela un combate por turnos entre equipos de unidades sobre un campo 2D discreto. Una **unidad** tiene salud, ataque, defensa, velocidad, una posición y, opcionalmente, *tags* que dicen de qué clase es (`undead`). Una **habilidad** es un cuerpo de sentencias que una unidad aplica sobre un objetivo; un **efecto** es lo que le ocurre, al comenzar cada uno de sus turnos, a la unidad que lo porta durante una cantidad de turnos (veneno, curación gradual). Cada unidad tiene un comportamiento `on turn` que decide qué hace cuando le toca: moverse, usar habilidades, preguntar por las unidades cercanas.

El tiempo se divide en **rondas**, y en cada una actúan las unidades vivas por orden de velocidad. Elegimos turnos y no tiempo real porque así cada batalla es una secuencia exacta de eventos, fácil de repetir y de probar; el espacio y los ataques de área no dependen de eso. Lo aleatorio (los dados `2d6`, las cantidades y las posiciones de los grupos) sale de un generador con semilla, así que una misma batalla es **repetible**, y variando la semilla se pueden probar muchas. Ese es el uso previsto: ajustar los números de un juego (daños, vidas, velocidades) y ver qué pasa, en lugar de simularlo a mano.

Quedan fuera las variables asignables: los únicos nombres de un cuerpo son los que fija el lenguaje (como `self` o `allies`), el parámetro de la habilidad o del efecto y la variable de cada `for`. También quedan fuera el costo y el enfriamiento de las habilidades, y su alcance: son extensiones razonables (Sección 4) que no hacían falta para tener un compilador completo y probado.

2.2 Lenguaje

Un programa es una secuencia de declaraciones en cualquier orden, y las sentencias no llevan terminador. El Listado 1 muestra casi todas las construcciones: una unidad con sus habilidades, un ataque de área con `radius`, un efecto, un comportamiento con condicionales y un equipo con un grupo aleatorio colocado alrededor de un punto.

Las expresiones tienen cinco tipos (`int`, `bool`, `string`, `unit` y `units`), con aritmética, comparaciones, `and`, `or`, `not`, acceso a atributos (`self.health`), dados (`2d6`), `radius r of c` (las unidades alrededor de `c`) y `nearest(ally)` o `nearest(enemy)`. Las palabras reservadas no se pueden usar como nombres, los identificadores son ASCII y los números tienen hasta 9 dígitos.

Respecto del Stage I, el lenguaje cambió con la devolución de la cátedra y al construir el frontend: los atributos se escriben completos (`health`, no `hp`), se eliminó el `;`, una habilidad pertenece a las unidades que la listan y puede restringir sus objetivos por relación o por tag, los efectos se declaran con `effect`, y un combate admite cualquier cantidad de equipos. El combate sigue siendo una declaración explícita (`battle`) porque elige qué equipos pelean: un programa puede

declarar más equipos de los que usa, y sin `battle` solo se valida. La gran adición es el **espacio**: posiciones, `arena`, `move`, `radius` para habilidades de área y grupos de unidades idénticas (`Archer * 20 around (5, 10)`).

```
arena (40, 20)

unit Hero at (0, 10) {
    health: 100, attack: 15, defense: 5, speed: 10,
    abilities: [Slash, Fireball]
}
unit Goblin { health: 12, attack: 5, defense: 1, speed: 4, abilities: [Spear] }
ability Slash on target: enemy {
    deal self.attack - target.defense to target
    apply Poison to target for 3
}
ability Fireball on target: enemy {
    for (victim in radius 5 of target) { if (victim != self) { deal 2d6 to victim } }
}
ability Spear on target: enemy { deal self.attack + 1d4 - target.defense to target }
effect Poison on victim { deal 2 to victim }

on turn Hero {
    if (self.health < 30) { move away from nearest(enemy) }
    else {
        move toward nearest(enemy)
        if (1d3 == 3) { use Fireball on nearest(enemy) } else { use Slash on nearest(enemy) }
    }
}
on turn Goblin { use Spear on nearest(enemy) }

party Heroes = [Hero]
encounter Goblins = [Goblin * 2d6 around (35, 10)]
battle: [Heroes, Goblins]
```

Listado 1. Un programa del lenguaje (la sintaxis completa está en el README).

3 Implementación

El compilador es una tubería: el frontend transforma el programa en un árbol de sintaxis abstracta (AST); el análisis semántico lo valida; la simulación ejecuta la batalla y la registra como una **traza** de eventos; y dos generadores, uno de texto y otro de HTML, producen el resultado a partir de la traza y de nada más (Figura 1). Todo lo que se escribe en la salida estándar es el resultado; los diagnósticos van a la salida de error. Partimos del proyecto base de la cátedra, del que conservamos la infraestructura (analizador *push*, *logger*, destructores, scripts, Docker e integración continua), y reemplazamos el frontend y el backend del ejemplo aritmético por los de nuestro lenguaje.

3.1 Frontend

Flex reconoce un lexema, lo convierte en un *token* y se lo empuja a Bison (`yypush_parse`), que mantiene su estado entre llamadas [4, 5]. Algunas decisiones:

- **Léxico.** Las palabras reservadas se escriben antes que el patrón de identificador, porque cuando dos patrones reconocen el mismo lexema Flex elige el primero: así `unit` es reservada. En cambio, `units` es un identificador por la regla del lexema más largo. Un número tiene hasta 9 dígitos, para que la conversión nunca desborde; un patrón adicional atrapa los más largos y da un mensaje preciso. Las cadenas no tienen escapes, lo que simplifica incluirlas luego en la salida HTML.
- **Sintaxis.** La gramática tiene 207 estados LALR(1) y ningún conflicto. Para poder prescindir del `;`, toda sentencia empieza con una palabra reservada, de modo que el fin de

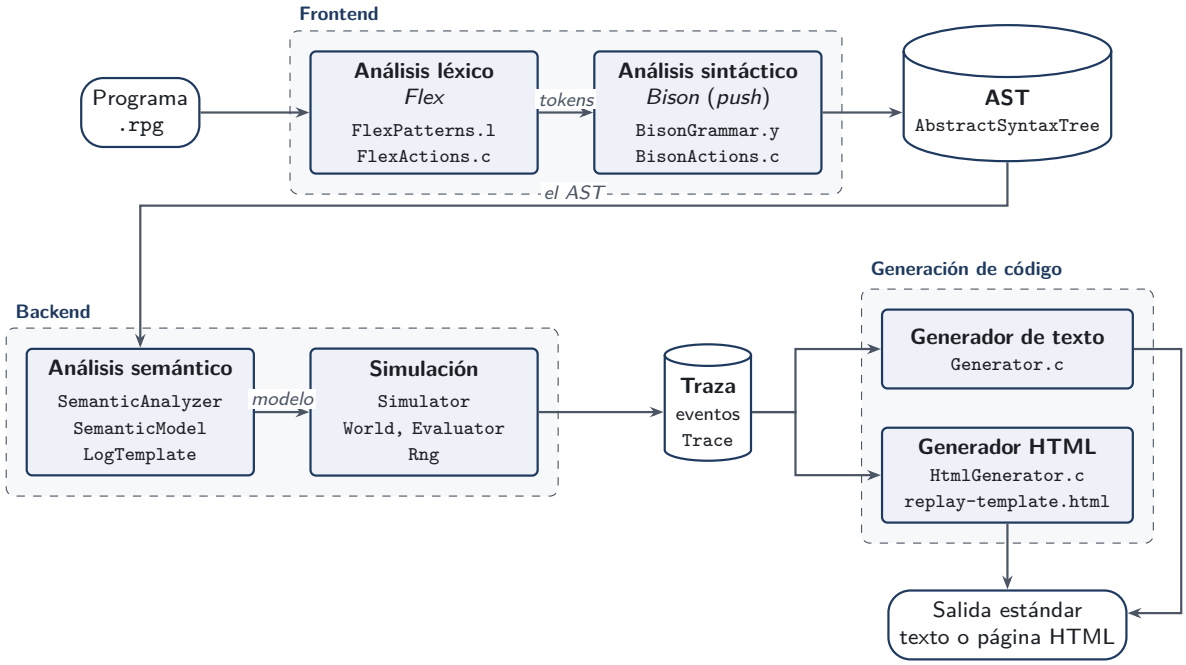


Figura 1. Arquitectura del compilador. Cada recuadro corresponde a uno o más archivos de `src/main/c`, salvo la plantilla HTML, que está en `src/main/resources`.

una y el comienzo de la siguiente nunca son ambiguos.

- **AST.** Un nodo por construcción, cada uno con la línea de su primer token, que es lo que permite informar `line 6: ...` en el análisis semántico sin tener el texto. Quien crea un nodo es su dueño hasta entregárselo a su padre, y los `%destructor` de Bison liberan lo que queda cuando el análisis falla. Como una cadena `1+1+...+1` produce un árbol tan profundo como larga es, la profundidad de una expresión está acotada al leer el programa (10000).

3.2 Backend

El backend son tres módulos encadenados (análisis semántico, simulación y generación), y cada uno sólo corre si el anterior aceptó el programa.

Artefacto final y *runtime*. En este dominio quien programa no quiere un ejecutable sino el **resultado de ejecutar su programa**: la batalla. Por eso el compilador incluye su propio *runtime*: la simulación es un intérprete sobre el AST ya validado, y la generación convierte lo que registró en el artefacto final, que puede ser un **registro de texto** o una **página HTML autocontenida**. El JavaScript de esa página es el único *runtime* que viaja con el artefacto, y sólo reproduce eventos ya resueltos, sin recalcular ninguna regla. Generar un programa en C para que el usuario lo compile habría exigido una cadena de herramientas y dos implementaciones de la misma semántica que tendrían que coincidir: la del compilador y la del código generado.

Análisis semántico. Recorre el AST e informa *todos* los errores, no sólo el primero, como `line N: mensaje` (hasta 100); un programa con errores se rechaza con estado 1. Valida:

- los nombres: únicos dentro de cada tipo de declaración (unidades, habilidades, efectos y equipos) y no reservados (`self`, `nearest`, …);
- las unidades: exactamente los cuatro atributos, escritos con literales, dados y aritmética (sin referencias a unidades), y que toda habilidad que listan exista;
- las habilidades: que cada tipo de objetivo sea `ally`, `enemy`, `self` o un tag declarado;
- los equipos, el combate y el campo: cantidades mayores que 0, a lo sumo un `arena` y un `battle` (con al menos dos equipos distintos), y posiciones dentro del campo;

- los cuerpos de habilidades, efectos y `on turn`: tipos (los cinco) y ámbitos (`self`, el parámetro, `allies`, `enemies` y la variable de cada `for`), y que un `use` dentro de `on turn` nombre una habilidad que esa unidad tiene.

Cada regla tiene un programa de prueba con un solo error y la lista exacta de mensajes esperados.

Simulación (mecanismo de evolución). Crea las unidades en orden y juega rondas: las unidades vivas actúan por velocidad (con empate, por orden de creación); en su turno una unidad deja actuar primero a sus efectos y luego ejecuta su `on turn`. Tras cada turno el combate termina si a lo sumo un equipo conserva unidades vivas (empate si ninguno), o al agotarse las rondas. El azar viene de SplitMix64 [3], escrito en el compilador en lugar de usar `rand`, para que la misma semilla dé la misma batalla en cualquier plataforma. Cada límite está declarado (unidades, eventos, iteraciones de `while`, profundidad de `use` y de bloques anidados), y excederlo detiene la simulación con estado 101 conservando lo ocurrido.

La traza y su generación. En lugar de escribir texto, la simulación registra una traza de eventos estructurados (inicio de ronda, movimiento, uso de habilidad, daño, curación, muerte, …) con los valores resultantes. Es la única interfaz hacia los generadores. El generador de texto produce el registro del Listado 2.

```
Shaman moves toward Knight#2: (19, 5) -> (16, 5)
Shaman uses Venom on Knight#2
  > Shaman spits venom at Knight#2
  Knight#2 is affected by Poison for 4 turns
Mage uses Fireball on Orc#2
  > Mage hurls a fireball at Orc#2!
  Knight#2 takes 5 damage (health 18/44)
  Orc#2 takes 8 damage (health 25/40)
  Orc#3 takes 7 damage (health 33/40)
[...]
Poison acts on Knight#2
  Knight#2 takes 3 damage (health 15/44)
```

Listado 2. Fragmentos de la salida de texto de `examples/01-skirmish.rpg` con `--seed 1`. Las consecuencias de cada acción van con sangría.

Con `--format html` el compilador escribe una sola página (Figura 2): una plantilla que el binario lleva embebida, en la que escribe la traza como JSON compacto. La página no usa nada externo, funciona sin conexión y la misma semilla da la misma página, byte a byte. Reconstruye el estado de la batalla en cualquier evento aplicando los valores registrados (por eso retroceder es inmediato) y calcula las métricas (daño y curación efectivos, muertes, por equipo y por unidad) sólo a partir de esos valores. Todo lo que escribe el programa del usuario (nombres, mensajes de `log`) es *dato*: el generador lo escapa y la página lo muestra siempre como texto, con una política de seguridad de contenido que no permite acceso a la red [6].

El compilador se invoca con `--seed`, `--max-rounds`, `--format text|html`, `--check` (valida y no genera nada) y `--syntax-only`. El estado de salida es `0` si el programa se acepta y la batalla termina, `1` si se rechaza, `2` si se agota la memoria, `3` si no se puede escribir la salida, `100` si la línea de comandos es inválida y `101` si la simulación falla. Los códigos propios empiezan en 100 porque el proyecto base reserva los anteriores.

3.3 Adicionales

- **Reproductor HTML con métricas.** Convierte la salida en algo que se puede mirar, recorrer y analizar (Figura 2).
- **Tres programas de ejemplo** (`examples/`) que muestran las construcciones principales del lenguaje (las pruebas las cubren todas) y terminan con un ganador para cada semilla de 1 a 20. Sirven de demostración y de pruebas.
- **Pruebas, integración continua y sanitizadores.** Hay 199 pruebas de unidad, más los

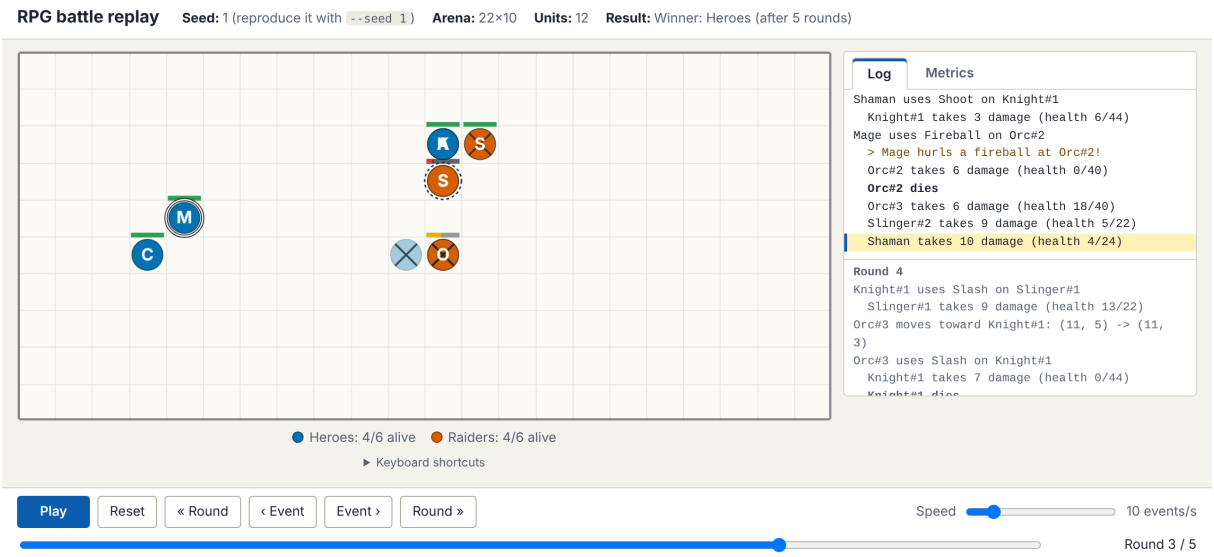


Figura 2. El reproductor con `examples/01-skirmish.rpg`: a la izquierda el campo, con las unidades de cada equipo (color, inicial y barra de salud; una cruz marca las que murieron); a la derecha el registro, con el evento actual resaltado, y abajo los controles. Hay una pestaña con las métricas.

3 ejemplos: cada prueba es el programa más corto posible para una sola característica, con la salida o la lista de mensajes exactos. El compilador se construye siempre con AddressSanitizer y UndefinedBehaviorSanitizer, y toda prueba falla ante cualquier reporte; GitHub Actions corre el mismo script en cada *push*. Además hicimos una pasada de robustez con programas mutados y generados a partir de los ejemplos y de las pruebas; sus hallazgos están en la sección siguiente.

3.4 Dificultades Encontradas

1. **Una gramática sin `;` y una precedencia que aceptaba el programa equivocado.** Sin terminador, una sentencia que puede empezar con una expresión se confunde con la anterior; lo resolvimos haciendo que todas empiecen con una palabra reservada. Peor fue `radius r of c`: con la precedencia del acceso con punto, `radius 5 of target.health` se leía como `(radius 5 of target).health`. Bison no informó ningún conflicto y, como en el Stage II todavía no había análisis semántico, el programa se aceptaba con un árbol distinto del buscado. Se resolvió dándole a `radius` su propio nivel de precedencia.
2. **Errores léxicos que perdían memoria.** Ante un carácter desconocido, el léxico abortaba sin que el parser liberara los fragmentos de AST ya construidos, y AddressSanitizer lo mostraba como una pérdida al rechazar un programa inválido. Ahora la acción empuja un token `UNKNOWN`: el parser falla y libera su pila.
3. **Definir la simulación sin ambigüedades.** «Cada unidad ataca al enemigo más cercano» deja abiertas preguntas que un programa no puede dejar abiertas: ¿`allies` incluye a quien actúa?, ¿qué pasa si el objetivo muere a mitad de un cuerpo?, ¿en qué orden actúan unidades con igual velocidad? Fijamos una regla para cada una, y cada regla tiene una prueba con salida exacta.
4. **Programas hostiles que hacían caer al compilador.** La pasada de robustez encontró desbordes de la pila de C: la recursión de `use` combinada con bloques anidados, y expresiones de cientos de miles de términos, cuyo árbol, igual de profundo, agotaba la pila al destruirse incluso con `--syntax-only`. Lo resolvimos con límites explícitos que fallan con un mensaje. También apareció un *log* de depuración de costo cuadrático, heredado del proyecto base: un identificador de 200000 letras tardaba siete segundos.
5. **Entornos distintos.** Desarrollamos en Windows y el compilador corre en un contenedor

Linux: Git convertía los finales de línea, y las pruebas, que comparan byte a byte, fallaban con archivos en CRLF. Se resolvió con reglas de `.gitattributes` que fuerzan LF.

4 Futuras Extensiones

- **Preguntar por las tags de una unidad.** Hoy una tag sólo restringe los objetivos de una habilidad; en `examples/02-three-armies` los Paladines distinguen a los no muertos por su velocidad. Una función `has_tag(unidad, tag)` encaja en el sistema de tipos actual.
- **Alcance, costo y enfriamiento de habilidades.** Una cláusula `range 3` haría que la habilidad se omita con un motivo (`out of range`); costo y enfriamiento requieren estado nuevo por unidad.
- **Variables y atributos modificables** (*buffs* y *debuffs*). Es el cambio más grande: exige un entorno mutable en la simulación y declaraciones en el análisis semántico.
- **Cortar un cuerpo cuando su unidad muere.** Hoy se ejecuta hasta el final.
- **Simulación por lotes.** Una opción `--runs N` con estadísticas agregadas (frecuencia de victoria de cada equipo, rondas promedio) para balancear sin relanzar el compilador.
- **Un índice espacial.** `nearest` recorre todas las unidades, y una batalla cuesta el cuadrado de su cantidad; una grilla lo reduciría a las celdas vecinas.
- **Mejores diagnósticos y un límite de tiempo real.** Informar la columna de un error y sugerir nombres, y una opción `--timeout`, porque los límites actuales acotan el trabajo pero no la duración.

5 Conclusiones

Recorrimos de punta a punta la construcción de un compilador para un lenguaje propio: del diseño del dominio y de la gramática a un resultado útil, una batalla repetible que se lee como texto o se mira en una página. Nos quedan tres lecciones. El diseño de la gramática condiciona el del lenguaje: casi toda decisión del frontend fue un compromiso entre cómo queríamos escribir y lo que Bison podía decidir con un solo token de anticipación. Una buena representación intermedia simplifica lo que viene después: que la simulación produzca una traza de valores nos dio dos generadores, un reproductor que retrocede sin recalcular y simulaciones que fallan sin perder lo ocurrido. Y las pruebas chicas con sanitizadores, más límites explícitos, hicieron del compilador algo robusto frente a programas hostiles.

La limitación principal es la expresividad (sin variables asignables, ni alcance o costo de habilidades). Es deliberada, para llegar a un compilador completo y probado, y la separación en fases y la traza como interfaz permiten agregar esas extensiones sin rehacer el resto.

6 Referencias

[1] Cátedra de Autómatas, Teoría de Lenguajes y Compiladores (2026). *Proyecto Especial: Diseño e Implementación de un Lenguaje* (6 de agosto de 2026). ITBA, Ingeniería Informática.

[2] Capridi, A. y Ametller, T. (2026). *Especificación: Diseño e Implementación de un Lenguaje* (Stage I). Grupo G-138, ITBA. Versionada como `doc/Especificación.pdf` en el repositorio.

[3] Steele, G. L., Jr., Lea, D. y Flood, C. H. (2014). Fast splittable pseudorandom number generators. En *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA '14)*, pp. 453–472. ACM. https://doi.org/10.1145/2660193.2660195

[4] *Flex: manual*. https://westes.github.io/flex/manual/

[5] *Bison: manual del Proyecto GNU*. https://www.gnu.org/software/bison/manual/

[6] W3C. *Content Security Policy Level 3*. https://www.w3.org/TR/CSP3/

7 Bibliografía

Material consultado que no se cita en el texto:

- Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.ª ed.). Addison-Wesley.
- Levine, J. R. (2009). *flex & bison*. O'Reilly Media.
- Nystrom, R. (2021). *Crafting Interpreters*. Genever Benning.